

API Data Fetcher with Local Cache

Assignment 8 — REST API + FutureBuilder + SharedPreferences Cache

Name: Soumitra Deshpande

Roll No: 150096724035

Cohort: Elon Musk

Environment: Flutter 3.x / Dart 3.x

Code: https://github.com/SoumitraDeshpande11/assignments/tree/assignments_8

1. Objective

Fetch data from a public REST API (JSONPlaceholder), display it with FutureBuilder, and cache the last successful result using SharedPreferences so the app still shows content when the network is unavailable.

2. Requirements Checklist

Requirement	Where it lives
Public REST API	https://jsonplaceholder.typicode.com/posts?_limit=12 via the http package
FutureBuilder	PostsScreen builds loading / error / data states from a Future<FetchResult>
SharedPreferences cache	PostsRepository._saveCache / _readCache store posts + timestamp as JSON
Data model class	Post with fromJson / toJson for API parsing and cache serialization
Refresh	App-bar refresh icon and a Retry button both re-run the future via setState

3. How Fetching Works

The Post class models one JSONPlaceholder post (id, title, body) with fromJson/toJson converters. PostsRepository.loadPosts() does the work:

1. http.get the JSONPlaceholder /posts endpoint.
2. On HTTP 200, decode the JSON array into Post objects, **save them to the cache**, and return them marked as live data.

3. On any failure — non-200 status, timeout, no network — it **falls back to the cache**: if a previous result was saved, it returns those posts marked `fromCache` with the original fetch time.

4. Only if there is no cache at all does the error reach the UI.

4. How the Cache Works

`SharedPreferences` stores simple key-value data that survives app restarts (on the web it uses `localStorage`):

```
await prefs.setString('posts_cache', jsonEncode(posts.map((p) =>
p.toJson()).toList()));
await prefs.setString('posts_cache_time',
DateTime.now().toIso8601String());
```

Reading reverses it: decode the JSON string back into `Post` objects and parse the saved timestamp. A banner at the top tells the user what they're looking at — "**Live data • fetched at HH:MM:SS**" in green, or "**Offline — cached data from HH:MM:SS**" in amber.

5. How FutureBuilder Works

`FutureBuilder` takes the future and rebuilds as it moves through states:

- **waiting** → spinner + "Fetching posts..."
- **error** → offline icon, the error message, and a Retry button
- **hasData** → the source banner plus a `ListView.builder` of post cards

The future is created once in `initState`; pressing Refresh or Retry calls `setState` with a brand-new future, which re-runs the fetch.

6. Widget Map

```
MaterialApp
├─ Scaffold
│   └─ AppBar ('JSONPlaceholder Posts' + refresh IconButton)
│       └─ FutureBuilder<FetchResult>
│           ├── waiting → CircularProgressIndicator
│           ├── error   → offline icon + message + Retry button
│           └─ data     → Column
│               ├── Container ← live/cached banner with timestamp
│               └─ Expanded
│                   └─ ListView.builder
│                       └─ Card per post (id avatar, title, body preview)
```

7. Source Code

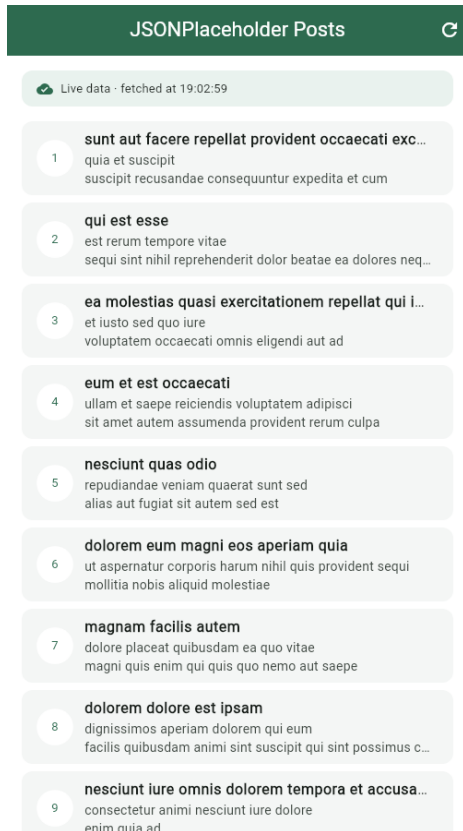
```
lib/main.dart

1 import 'dart:convert';
2
3 import 'package:flutter/material.dart';
4 import 'package:http/http.dart' as http;
5 import 'package:shared_preferences/shared_preferences.dart';
6
7 void main() => runApp(const FetcherApp());
8
9 class Post {
10   const Post({required this.id, required this.title, required this.body});
11
12   final int id;
13   final String title;
14   final String body;
15
16   factory Post.fromJson(Map<String, dynamic> json) =>
17     Post(id: json['id'], title: json['title'], body: json['body']);
18
19   Map<String, dynamic> toJson() => {'id': id, 'title': title, 'body': body};
20 }
21
22 class FetchResult {
23   const FetchResult({
24     required this.posts,
25     required this.fromCache,
26     required this.fetchedAt,
27   });
28
29   final List<Post> posts;
30   final bool fromCache;
31   final DateTime fetchedAt;
32 }
33
34 class PostsRepository {
35   static const String _cacheKey = 'posts_cache';
36   static const String _cacheTimeKey = 'posts_cache_time';
37   static const String _apiUrl =
38     'https://jsonplaceholder.typicode.com/posts?_limit=12';
39
40   Future<FetchResult> loadPosts() async {
41     try {
42       final response = await http.get(Uri.parse(_apiUrl));
43       if (response.statusCode != 200) {
44         throw Exception('HTTP ${response.statusCode}');
45       }
46       final posts = (jsonDecode(response.body) as List)
47         .map((e) => Post.fromJson(e))
48         .toList();
49       await _saveCache(posts);
50       return FetchResult(posts: posts, fromCache: false, fetchedAt: DateTime.now());
51     } catch (e) {
52       final cached = await _readCache();
53       if (cached != null) return cached;
54       rethrow;
55     }
56   }
57
58   Future<void> _saveCache(List<Post> posts) async {
59     final prefs = await SharedPreferences.getInstance();
60     prefs.setString(_cacheKey, jsonEncode(posts));
61     prefs.setString(_cacheTimeKey, DateTime.now().toIso8601String());
62   }
63
64   Future<FetchResult> _readCache() async {
65     final cachedPosts = jsonDecode(prefs.getString(_cacheKey));
66     if (cachedPosts == null) return null;
67     final cachedPostsList = List<Post>.of(cachedPosts.map((e) => Post.fromJson(e)));
68     final cachedTime = prefs.getString(_cacheTimeKey);
69     if (cachedTime == null) return null;
70     return FetchResult(posts: cachedPostsList, fromCache: true, fetchedAt: DateTime.parse(cachedTime));
71   }
72 }
```

8. App Screenshots

The app was built for the web (`flutter build web`) and captured in Chrome at two window sizes. Both show a **live fetch from JSONPlaceholder** — the green "Live data · fetched at ..." banner is visible above the post list.

Phone width (500px):



Desktop width (1280px):

JSONPlaceholder Posts

Live data · fetched at 19:03:02

1

sunt aut facere repellat provident occaecati excepturi optio reprehenderit
quia et suscipit
suscipit recusandae consequuntur expedita et cum

2

qui est esse
est rerum tempore vitae
sequi sint nihil reprehenderit dolor beatae ea dolores neque

3

ea molestias quasi exercitationem repellat qui ipsa sit aut
et iusto sed quo iure
voluptatem occaecati omnis eligendi aut ad

4

eum et est occaecati
ullam et saepe reiciendis voluptatem adipisci
sit amet autem assumenda provident rerum culpa

5

nesciunt quas odio
repudiandae veniam quaerat sunt sed
alias aut fugiat sit autem sed est

6

dolorem eum magni eos aperiam quia
ut aspernatur corporis harum nihil quis provident sequi
mollitia nobis aliquid molestiae

7

magnam facilis autem
dolore placeat quibusdam ea quo vitae
magni quis enim qui quis quo nemo aut saepe

8

dolorem dolore est ipsam
dignissimos aperiam dolorem qui eum

9. Terminal

Real build-and-run session: `flutter pub get`, `flutter analyze` (no issues), `flutter build web`, then `flutter run -d web-server` to serve the app locally.

```
zsh - assignment_8

$ flutter --version
Flutter 3.47.2 • channel stable • https://github.com/flutter/flutter.git
Framework • revision d3b14c8769 (3 weeks ago) • 2026-08-26 16:07:51 -0700
Engine • hash 1cf1c4773fb941c4c74a7f8bb144a8837596c0f4 (revision a804b26164)
Tools • Dart 3.13.2 • DevTools 2.60.0

$ flutter pub get
Got dependencies!

$ flutter analyze
No issues found! (ran in 3.2s)

$ flutter build web --release
Compiling lib/main.dart for the Web... 22.9s
✓ Built build/web

$ flutter run -d web-server --web-port=8748
Launching lib/main.dart on Web Server in debug mode...
Waiting for connection from debug service on Web Server... 10.9s
lib/main.dart is being served at http://localhost:8748

Flutter run key commands.
r Hot reload.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).
```

10. Run Instructions

```
flutter pub get
```

```
flutter run          # pick a device, or use flutter run -d chrome
```

The app fetches 12 posts on launch. Tap the refresh icon to fetch again. To see the cache in action, load the app once, go offline (or block the network), and relaunch — the posts reappear from the cache with the amber offline banner.

11. Possible Extensions

- Add a "stale after N minutes" rule so old cache entries trigger a background refresh.
- Cache images too, using `cached_network_image`.
- Paginate with `/posts?_page=N` and cache each page.

Built by Soumitra Deshpande • Roll No. 150096724035 • Cohort: Elon Musk